

COMSATS UNIVERSITY ISLAMABAD

Department of Economics

ASSIGNMENT 02
CSC-211
DATA STRUCTURES AND ALGORITHMS

Magic Square Simulation and Algorithm Analysis

Prepared for
Dr. Omer Ahmed
Course Instructor
COMSATS University Islamabad

Prepared by

Raja Aashir Mehmood
BED-4A
Economics with Data Science

Magic Square Simulation and Algorithm Analysis

Prepared for

Dr. Omer Ahmed
Course Instructor
COMSATS University Islamabad

Submitted on

November 01, 2026
Monday

Magic Square Simulation and Algorithm Analysis

Table of Contents

1. Introduction:.....4
Magic constant:.....4
1.1. Project overview:.....4
2. Constructive algorithms:.....6
2.1. Odd order: Siamese method:.....9
2.2. Doubly even order: diagonal/complement method.....11
2.3. Singly even order: Strachey's method:.....12
3. Brute Force Method:.....14
3.1. Naive brute force:.....14
3.2. Backtracking with pruning:.....15
3.3. Keeping the window responsive.....16
4. Time-complexity comparison:.....16
4.2. Naive brute force: O((n²)!).....17
4.3. Backtracking: worst case and why pruning helps:.....18
4.4. Measured results:.....18
4.5. When does brute force become impractical?.....19
4.6. Summary of the complexities:.....20
5. Challenges:.....21
6. AI usage reflection:.....22
7. Conclusion:.....23
References.....24

1. Introduction:

A ***MAGIC SQUARE*** of order n is a square grid with n rows and n columns containing every integer from 1 to n^2 exactly once. Its defining property is that every row, every column, and the two main diagonals have the same total. This shared total is known as the magic constant. A valid magic square must satisfy two conditions at the same time: no value from 1 to n^2 may be missing or repeated, and every required line must have the same sum. A grid that merely looks balanced, but fails either condition, is not a valid magic square.

Magic constant:
The numbers from 1 to n^2 have a total of $n^2(n^2 + 1)/2$. Since this total is divided equally among n rows, the magic constant is:

$$\mathbf{M(n) = n(n^2 + 1) / 2}$$

Examples: M(3) = 15, M(4) = 34, M(5) = 65, and M(6) = 111.

Three cases and their algorithms:

Case	Order condition	Examples	Construction method
Odd	n is odd	3, 5, 7, 9, …	Siamese (de la Loubère)
Doubly even	n mod 4 = 0	4, 8, 12, …	Diagonal/complement method
Singly even	n mod 4 = 2	6, 10, 14, …	Strachey's method

Order 2 has no magic square. Order 1 is trivial, but the application focuses on orders $n \geq 3$. The program classifies the order and selects the matching construction method.

1.1. Project overview:

Ordematrix is a Python desktop application that uses Tkinter to provide an interactive graphical user interface. The user selects an order and a solving approach, and the program identifies the appropriate case before constructing or searching for a square. Its constructive algorithms are intended to demonstrate the mathematical procedure directly rather than guess a solution. The simulation can show the grid being filled, while the algorithm panel helps connect the visual result to the steps of the method. The application also includes brute-force options for comparison, timing and search-count information, and a verifier that checks the finished result. These features make the project both a programming exercise and a way to study algorithm efficiency.

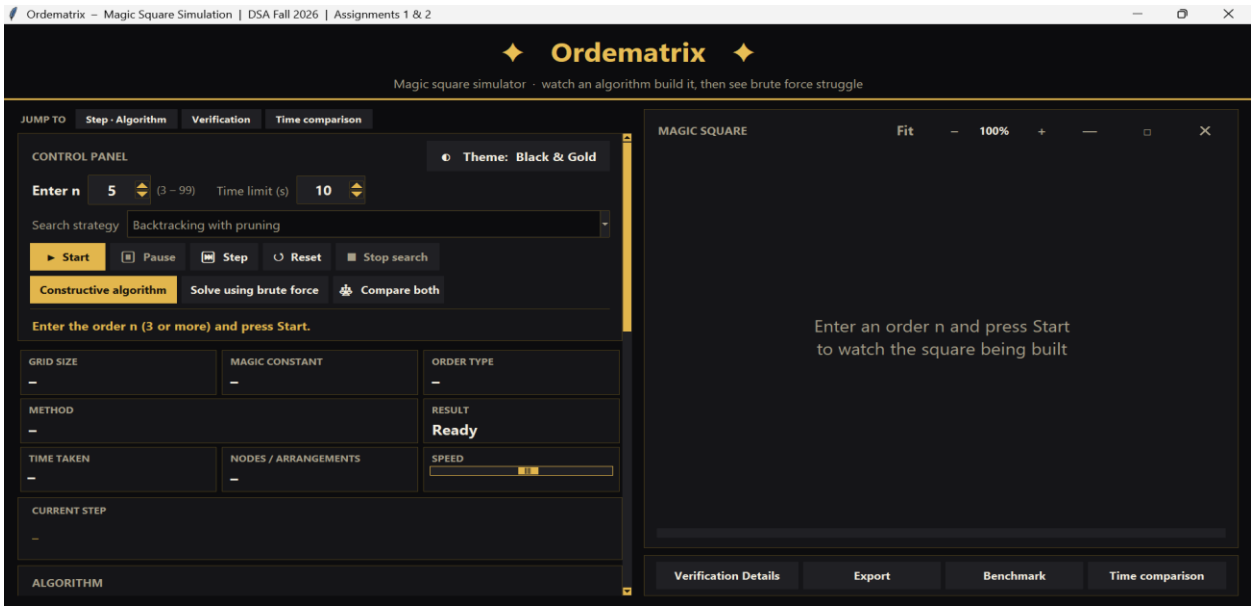
The main objectives of this project are to

- Implement and explain the three constructive methods.

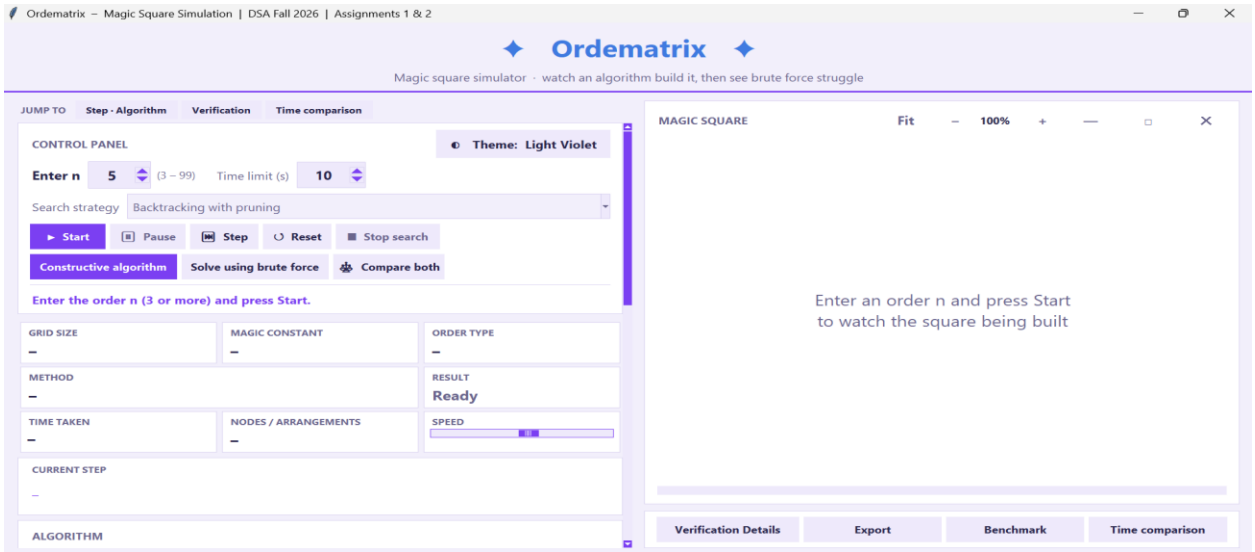
- Present each method through a graphical simulation.
- Verify that each generated square satisfies the mathematical definition.
- Compare constructive algorithms with naive brute force and backtracking with pruning.
- Use observed timing and search counts to discuss scalability.

The report separates theoretical complexity from measured performance because actual timings depend on the computer, Python version, system load, and the implementation details.

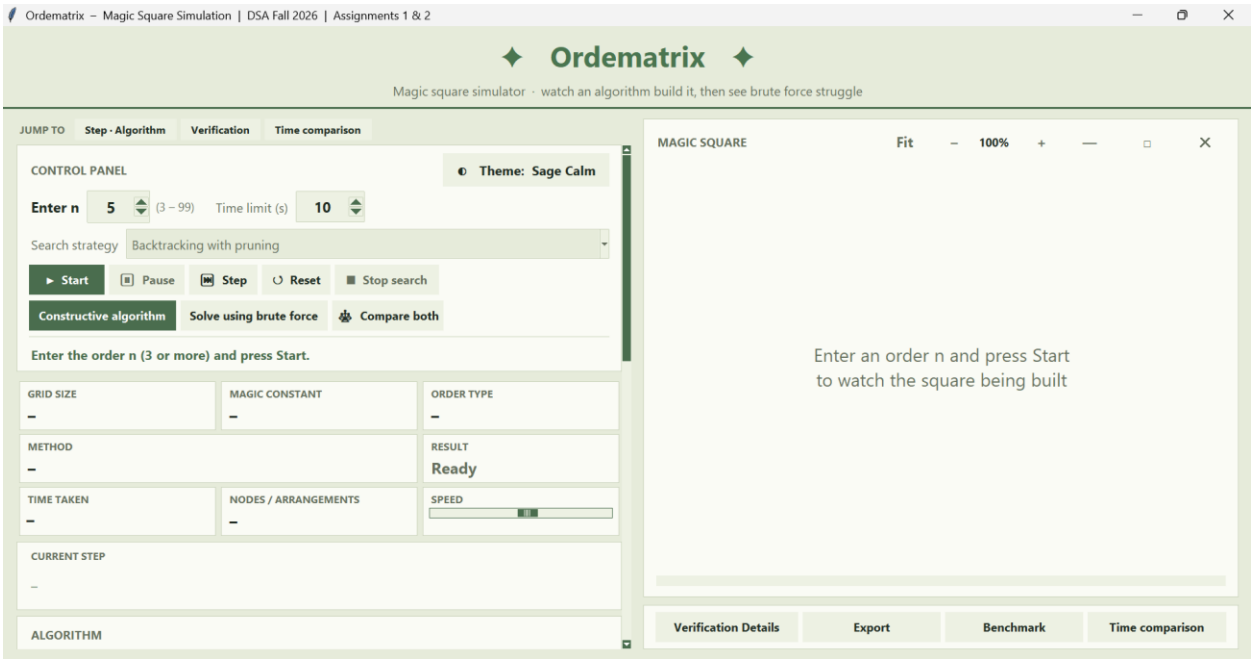
Layout 01:



Layout 02:



Layout 03:



2. Functionality of Ordematrix:

Ordematrix constructs, animates and verifies magic squares of every order from 3 to 99. It builds odd orders with the Siamese method, doubly even orders with the diagonal/complement method and singly even orders with Strachey's method. It compares these with two brute-force searches, naive permutations and backtracking with pruning. The window has two halves. The left half holds the controls, live statistics and information panels, and scrolls on its own. The right half is a full-height magic square card with the tool buttons beneath it. A Theme button cycles through Black & Gold, Light Violet and Sage Calm, and the grid, charts, tables and pop-ups all change together.

Control panel

Control	**Function**
Enter n	Order of the square, 3 to 99, typed or set with ▲ ▼. Grid size, magic constant, order type and method appear as soon as n is typed.
Time limit (s)	Longest a brute-force search may run (1–3600 s, default 10). Compare both uses the same limit.
Search strategy	Backtracking with pruning or Naive permutations. Active only in brute-force mode.
Constructive algorithm / Solve using brute force	Chooses the solving method. The selected button is highlighted.
Start (or Enter)	Validates the input and builds the square step by step. A finished run is replayed from the start.
Pause / Step / Reset	Pause freezes the animation, Step advances exactly one step,

	Reset clears the grid.
Speed slider	Slow to instant. At the top speeds several steps are drawn per tick.
Stop search	Cancels a running search or comparison. Active only while one is running.
Compare both	Times the constructive algorithm and both searches on the same n and fills the Time Comparison panel.
Message line	Shows instructions, progress and errors.

Controls that would interfere with a running search are disabled until it finishes. When brute force finds a square, it is replayed row by row, so it can be watched like the constructive methods.

Live statistics

Eight cards update during every run. They show:

- grid size
- magic constant
- order type
- method
- result (Ready, Searching…, Valid Magic Square ✓, Limit reached, Stopped and so on)
- time taken, in a read-only box measured with time.perf_counter()
- nodes/arrangements examined
- the speed slider

During a search, the elapsed time and node count refresh every 100 ms.

Magic square card

- **Display:** The grid has a gold border, and cell colours shade from low to high numbers. The cell being filled is highlighted, and a progress bar shows how far the simulation has got.
- **Readable numbers:** Every number stays visible, even for large n. Instead of shrinking the cells, the grid scrolls, and scroll bars appear only when needed.
- **Zoom:** The zoom range is 10% to 800%, using the + / − / Fit buttons, Ctrl + mouse wheel, Ctrl + / Ctrl −, Ctrl 0, or a double-click on the grid. The grid can be panned by dragging.
- **Window buttons:**
  - **— (minimise):** collapses the grid.
  - □ **(maximise):** hides the banner and left column and gives the square the whole window. A strip then shows the app name, grid size, magic sum and method, and 100% means the whole square fits. Esc restores the normal view.
  - ✕ **(close):** hides the square, and “Reopen magic square” brings it back.

Information panels

- **Current step:** a plain-English explanation of each step, such as "up-right cell is occupied, so move one cell down".
- **Algorithm:** pseudocode for the chosen method (Siamese, complement, Strachey, naive or backtracking). The line being executed is highlighted as the simulation runs.
- **Verification:** shows M(n), whether every row, column and both diagonals add up to it, whether 1…n² are each used once, and the verdict "VALID MAGIC SQUARE". When the grid is complete, each line is highlighted in turn on the grid.
- **Time comparison:** log-scale bars and a table of time, nodes, status and "compared with algorithm", plus a summary sentence. It fills in automatically after any normal run, and Download report saves it as CSV, PDF or TXT.

JUMP TO buttons above the left column (Step · Algorithm, Verification, Time comparison) slide smoothly to each section, and the column also jumps to Verification when a run finishes.

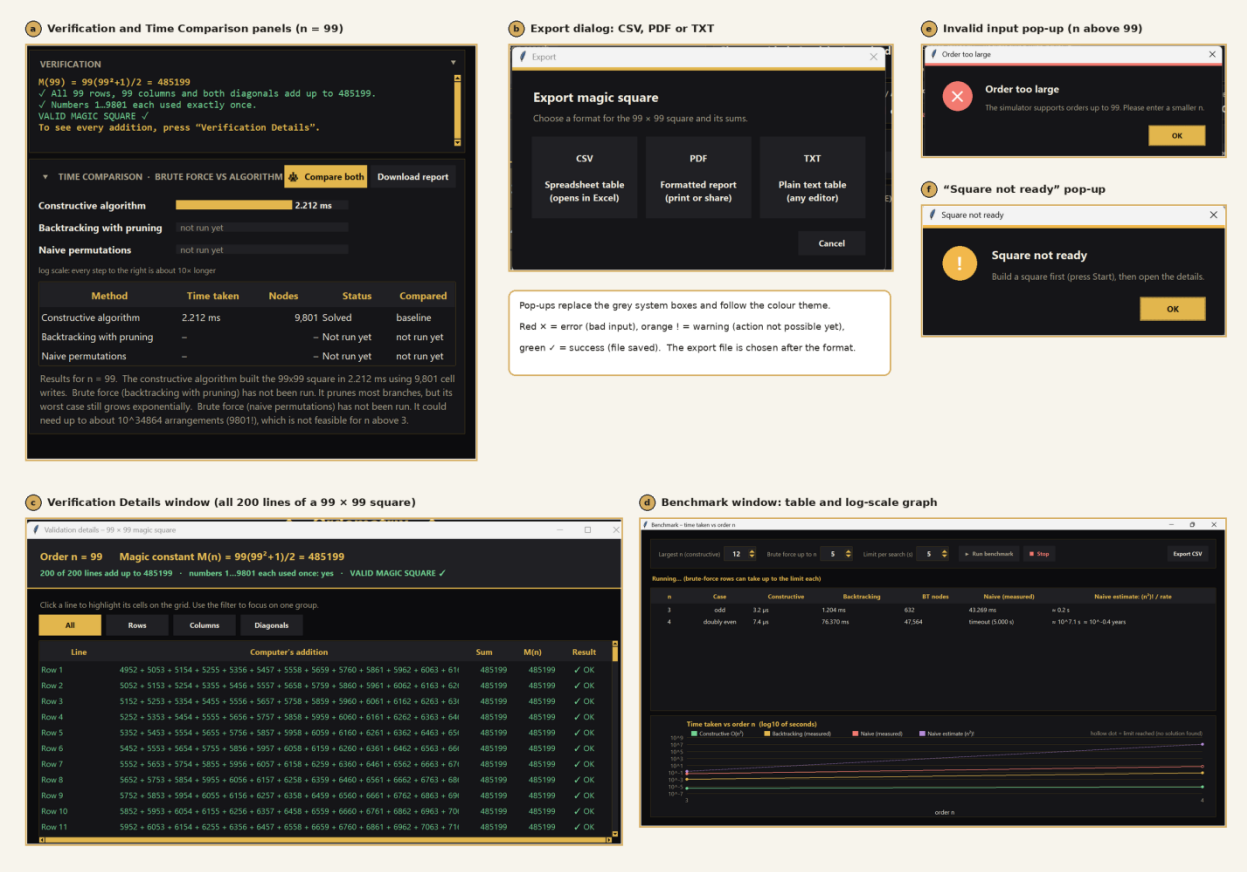
Tools and reports

Button	**Function**
Verification Details	A table of every row, column and diagonal with the exact addition (e.g. 8 + 1 + 6 = 15), running total, M(n) and ✓/✗. It can be filtered by All, Rows, Columns or Diagonals, and clicking a line highlights its cells on the grid.
Export	Saves the finished square as CSV (opens in Excel), PDF (formatted report with the coloured square and all line sums) or TXT (aligned tables).
Benchmark	Measures time against n for all methods and shows a table, a log-scale graph and an estimate for naive search on larger n. It exports CSV.
Time comparison	Slides to the comparison panel.

Input validation and responsiveness

Themed pop-ups explain each problem instead of crashing. They cover an empty box, non-numbers, n = 1, n = 2 (no such square), n < 1, n above 99 and an invalid time limit. Brute force runs in a worker thread, and the search ends with one of four statuses: found, time limit reached, cancelled or error. The window stays responsive throughout. Other conveniences include hover tooltips, mouse-wheel scrolling of the left column and sharp text on high-resolution Windows screens.

Functionalities Screenshots:



3. Constructive algorithms:

we state the exact rule that our program uses and that we checked by testing.

3.1. Odd order: Siamese method:

Rules

- The order n must be an odd integer, $n \geq 3$.
- Put the number 1 in the middle cell of the top row.
- Put each next number one cell up and one cell to the right of the previous number.
- If the move leaves the top row, continue from the bottom row (wrap around).
- If the move leaves the right-most column, continue from the left-most column (wrap around).
- If the target cell is already taken, put the next number directly below the previous number instead.
- Continue until all the numbers 1 to n^2 are placed. Each number is used exactly once.

Pseudocode:

```
row ← 0; col ← n // 2
for k = 1 to n²:
   grid[row][col] ← k
   next ← ((row − 1) mod n, (col + 1) mod n)
   if grid[next] is occupied:
      next ← (row + 1, col)
   (row, col) ← next
```

Worked example: n = 3

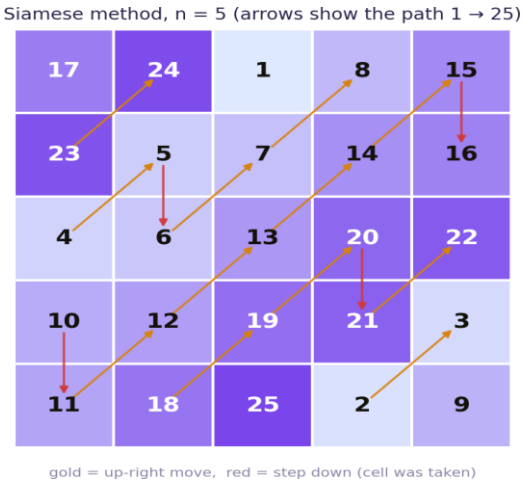
The magic constant is $M(3) = 3(3^2 + 1)/2 = 15$. The resulting square is:

k	Move	Cell used	k	Move	Cell used
1	start: top row, middle	(1, 2)	6	up-right from (2,2)	(1, 3)
2	up wraps to bottom, right	(3, 3)	7	up-right wraps to (3,1): taken, so go below 6	(2, 3)
3	up-right, right wraps to col 1	(2, 1)	8	up-right from (2,3), wraps to col 1	(1, 1)
4	up-right gives (1,2): taken, so go below 3	(3, 1)	9	up wraps to bottom, right	(3, 2)
5	up-right from (3,1)	(2, 2)			

8	1	6
3	5	7
4	9	2

Result. Rows: 8+1+6 = 3+5+7 = 4+9+2 = 15. Columns: 8+3+4 = 1+5+9 = 6+7+2 = 15. Diagonals: 8+5+2 = 15 and 6+5+4 = 15. This is the same square as in our notes [1][2] and in the output of our program.

Every row, column, and main diagonal sums to 15. The method places one number per cell and therefore takes $\Theta(n^2)$ time.



Odd-order simulation (n = 5) :

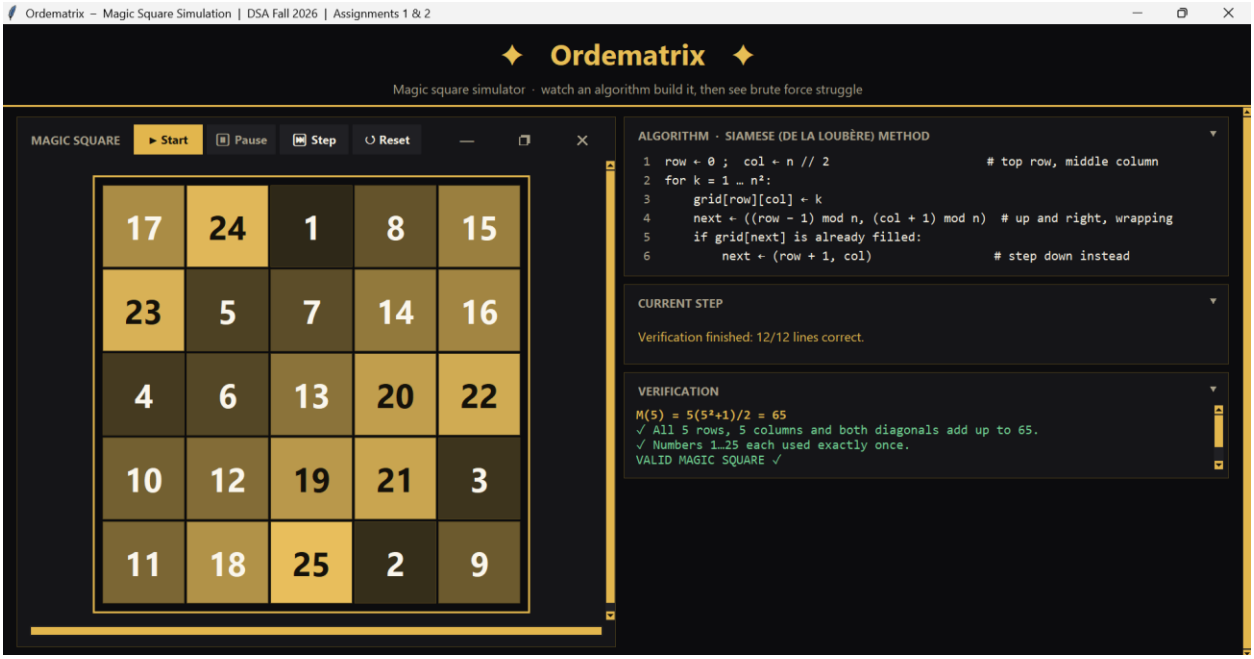


Figure: Odd-order simulation (n = 5). screenshot from Ordematrix App.

3.2. Doubly even order: diagonal/complement method

The diagonal/complement method applies to doubly-even orders, where n is divisible by 4.

Rules:

- The order n must be divisible by 4 (n = 4k). Examples are 4, 8 and 12.
- Fill the square with 1 to n^2 row by row (reading order).
- Divide the square in your mind into 4 × 4 blocks.
- In every 4 × 4 block mark the cells on the two diagonals (an "X" pattern). In program terms, a cell (i, j) is marked when (i mod 4 = j mod 4) or (i mod 4 + j mod 4 = 3).
- Replace the number x in each marked cell by its complement $n^2 + 1 - x$.
- Leave all other cells unchanged. Every number from 1 to n^2 is still used exactly once.
- Check that every row, column and both main diagonals add up to M(n).

Pseudocode

```
for each cell (i, j):
    grid[i][j] ← i*n + j + 1
for each cell (i, j):
    if (i mod 4 = j mod 4) or (i mod 4 + j mod 4 = 3):
        grid[i][j] ← n² + 1 − grid[i][j]
```

Worked example: n = 4

Here M(4) = 4(16 + 1)/2 = 34. Phase 1 writes 1 to 16 in reading order. Phase 2 marks the eight cells of the X pattern (the four corners and the four middle cells) and complements them, for example 1 → 17 − 1 = 16, 4 → 13, 6 → 11 and 7 → 10.

16	2	3	13
5	11	10	8
9	7	6	12
4	14	15	1

Check. Rows: 16+2+3+13 = 5+11+10+8 = 9+7+6+12 = 4+14+15+1 = 34. Columns: 16+5+9+4 = 2+11+7+14 = 3+10+6+15 = 13+8+12+1 = 34. Diagonals: 16+11+6+1 = 34 and 13+10+7+4 = 34. The square in the notes [4], [[1,15,14,4],[12,6,7,9],[8,10,11,5],[13,3,2,16]], uses the other convention and also has M = 34.

Doubly-even simulation (n = 4) :

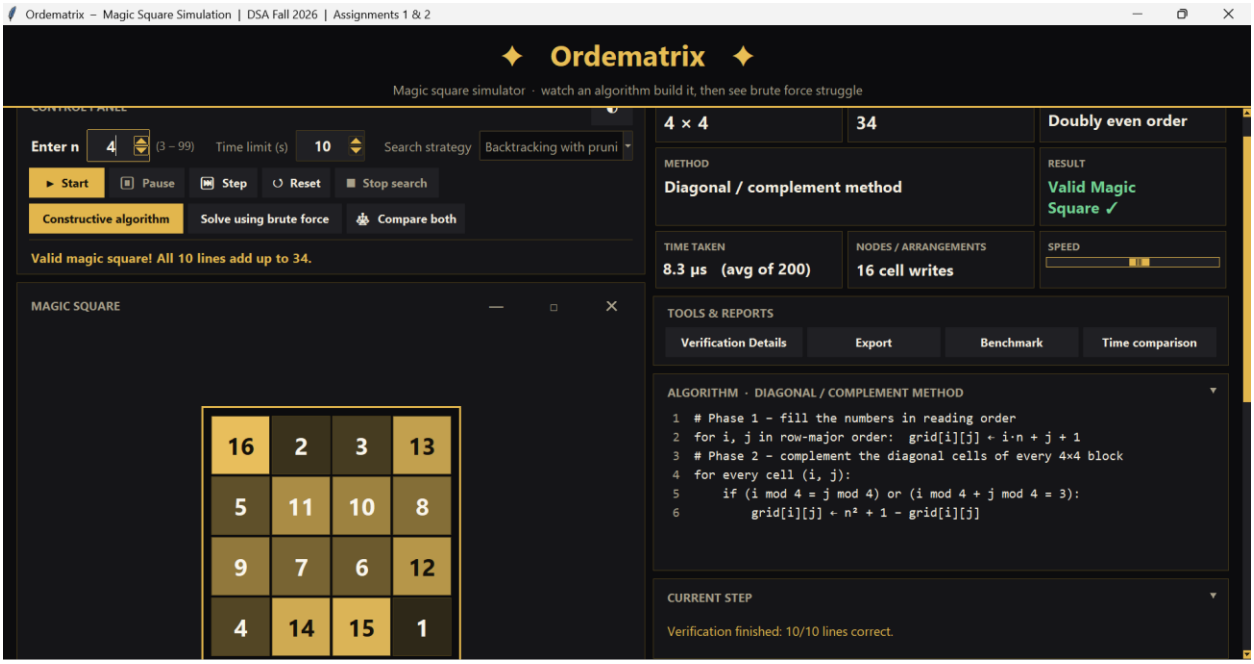


Figure: Doubly-even simulation (n = 4). Screenshot from Ordematrix.

3.3. Singly even order: Strachey's method:

A singly-even order has the form n = 4k + 2, such as 6, 10, or 14. Strachey's method begins with an odd magic square

Rules:

- The order must be n = 4k + 2 with k ≥ 1, so n = 6, 10, 14, … (there is no magic square of order 2).
- Let m = n/2. Because n = 4k + 2, m = 2k + 1 is odd.
- Build an odd magic square A of order m with the Siamese method.

- Divide the n × n square into four quadrants of size m × m. Fill them with copies of A plus an offset: top-left = A, bottom-right = A + m^2, top-right = A + $2m^2$, bottom-left = A + $3m^2$.
- Let k = (m − 1)/2. Exchange the first k columns of the top-left quadrant with the same columns of the bottom-left quadrant. The exception is the middle row of the top half: there, the exchange starts one column to the right (columns 2 to k + 1), so the
- middle cell of the first column stays where it is and the central cell is exchanged.
- Exchange the last k − 1 columns of the top-right quadrant with the same columns of the bottom-right quadrant. (For n = 6 we have k = 1, so there is nothing to exchange here.)
- Every number from 1 to n^2 now appears exactly once.
- Check that every row, column and both main diagonals add up to M(n).

Pseudocode

```
m ← n / 2; k ← (m − 1) / 2
A ← SiameseMagicSquare(m)
Place A and offset copies in four quadrants
Swap the prescribed left-side columns
Apply the middle-row exception
Swap the prescribed right-side columns
Verify all rows, columns, and diagonals
```

Worked example: n = 6

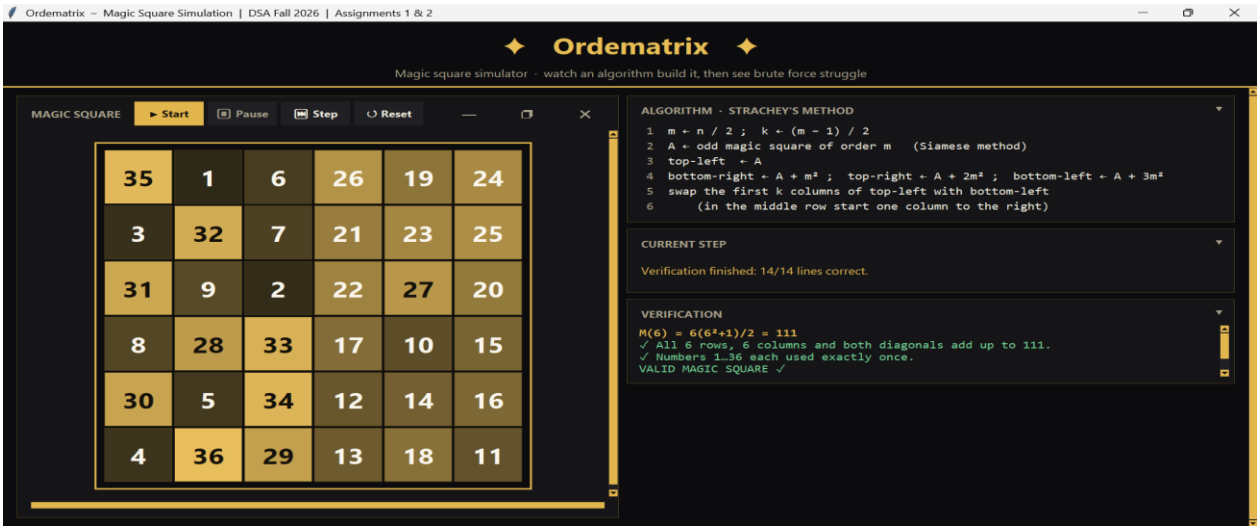
Check (M = 111). Row 1: 35+1+6+26+19+24 = 111.

35	1	6	26	19	24
3	32	7	21	23	25
31	9	2	22	27	20
8	28	33	17	10	15
30	5	34	12	14	16
4	36	29	13	18	11

31+9+2+22+27+20	=	111.	Row	4:
8+28+33+17+10+15	=	111.	Row	5:
30+5+34+12+14+16	=	111.	Row	6:
4+36+29+13+18+11	=	111.	Column	1:

Row 2: 3+32+7+21+23+25 = 111. Row 3:35+3+31+8+30+4 = 111, and the other columns are also 111. Main diagonal: 35+32+2+17+14+11 = 111. Anti-diagonal: 24+23+9+28+5+4 = 93+… see the program's "Verification Details" window for all 14 sums. This 6 × 6 square is identical to the Strachey example.

Singly even order: Strachey's method(n=6):



All three constructive algorithms have $\Theta(n^2)$ time complexity. The output itself contains n^2 cells, so a method that writes each cell at least once cannot run in less than $\Omega(n^2)$ time. Each method performs a constant amount of work per cell, with a limited number of additional passes or swaps, so its overall time is also $O(n^2)$. Their auxiliary space is $O(n^2)$ when the complete square is stored in a two-dimensional grid.

4. Brute Force Method:

The constructive methods know how to build a magic square. A brute-force method knows nothing about magic squares. It only knows how to check one, and it searches until it finds a square that passes the check. Our program offers two searches and lets the user choose between them. Both run in a worker thread.

4.1. Naive brute force:

The naive search generates every arrangement (permutation) of the numbers 1 to n^2, puts it into the grid row by row, and tests whether it is magic. It stops at the first magic square. For n = 3 there are 9! = 362,880 arrangements, which is small. For n = 4 there are 16! ≈ 2.1×10^{13}, which is far too many.

PseudoCode:

```
NaiveSearch(n, timeLimit):
   M ← n(n² + 1) / 2                  # magic constant
   nodes ← 0
   for each permutation P of 1, 2, …, n²:
      nodes ← nodes + 1
```

```
    if nodes mod 1024 = 0:          # periodic housekeeping
       if Stop was pressed:     return CANCELLED
       if elapsed ≥ timeLimit:  return LIMIT REACHED
    if sum of first n numbers of P ≠ M:
       continue                 # cheap early rejection, O(n)
    grid ← P written row by row into an n × n grid
    if every row sum = M and every column sum = M
      and main diagonal = M and anti-diagonal = M:
       return grid, nodes          # FOUND
  return NO RESULT
```

The quick first-row test throws away most arrangements after adding only n numbers, but it does not change the growth rate: the search still has to look at a huge number of arrangements. Every 1,024 arrangements the search updates a counter for the GUI and checks the Stop button and the time limit. If the limit is reached, the program reports “Limit reached” instead of hanging.

4.2. Backtracking with pruning:

Backtracking builds the square cell by cell in reading order. It tries the unused numbers in increasing order. If a partial grid can no longer become a magic square, the search abandons it at once (this is pruning) and goes back to change the previous choice (this is backtracking). The program keeps the running sum of every row, every column and both diagonals, and a sorted list of the numbers that are still free.

Pruning rules:

- ***Overflow rule***. A row, column or diagonal sum may never be larger than M(n). Because the free numbers are tried in increasing order, when one number is too big all the larger numbers are too big as well, so the loop stops (a b r e a k).
- ***Completion rule.*** The last cell of a row, column or diagonal must make the sum equal to M(n) exactly. Any other number is rejected. This means the last cell of every line is decided, not chosen.
- ***Reachability (bound) rule***. If k cells of a row (or column) are still empty, the sum so far plus the k smallest free numbers must not be more than M(n), and plus the k largest free numbers must not be less than M(n). Example for n = 4 (M = 34): if a row already has 16 and 15 (sum 31) and two cells are left, they must add to 3, which needs the numbers 1 and 2. If 1 and 2 are no longer free, the row cannot be finished, so the search backs up immediately.
- ***Diagonal rule.*** The same overflow and completion tests are applied to the main diagonal and the anti-diagonal.
-

Pseudocode:

```
place(pos):
   if pos = n²: return success
   for each unused v in ascending order:
```

```
if row/column/diagonal sum + v > M: break          # overflow rule
if last cell of a line and sum + v ≠ M: continue   # completion rule
if sum + k smallest free > M or sum + k largest free < M: continue   # reachability rule
write v; if place(pos + 1): return success
erase v                                  # backtrack
```

4.3. Keeping the window responsive

A long search must not freeze the window. Tkinter is not thread-safe, so we follow one rule: only the main thread touches the widgets. The search runs in a daemon thread and shares one small S e a r c h C o n t r o l object with the GUI. It holds the time limit, a "cancel" event set by the Stop button, and a live node counter. The GUI reads the counter and the elapsed time every 100 ms with a f t e r ( ) to update the screen, and the worker puts its final result into a queue that the GUI reads. A search ends with one of four statuses: found, time limit reached, cancelled, or error.

Both searches display the time taken and the number of arrangements (nodes) examined. The Compare both button runs the constructive algorithm and both searches on the same n and shows the results side by side with bars, a table, and a downloadable report.

5. Time-complexity comparison:

5.1. Constructive methods: $O(n^2)$:

The grid has n^2 cells, and each constructive method writes each cell a fixed number of times, so the time is proportional to n^2.

- **Siamese (odd).** The loop runs k = 1 … n^2. Each pass does one write, two modulo operations and one test, which is constant work. Total: $\Theta(n^2)$.
- **Doubly even.** Phase 1 writes n^2 numbers. Phase 2 visits n^2 cells and, for the marked ones, does one subtraction. The test uses two modulo operations. Total: $\Theta(n^2)$.
- **Singly even (Strachey).** The odd square of order m = n/2 costs $\Theta(m^2) = \Theta(n^2/4)$. Copying it into four quadrants costs $4m^2 = n^2$ writes. The swaps touch at most about $n^2/4$ cells. Total: $\Theta(n^2)$.

The output itself has n^2 numbers, so no method can be faster than $\Omega(n^2)$. The constructive methods are therefore optimal up to a constant. The verification step also costs $\Theta(n^2)$, because it adds every cell once for its row and once for its column, plus the two diagonals.

n	Case	Time	Time per cell
3	odd	1.0 µs	108 ns
4	doubly even	2.0 µs	128 ns
5	odd	2.0 µs	81 ns
6	singly even	3.6 µs	99 ns
8	doubly even	6.3 µs	98 ns

10	singly even	8.3 µs	83 ns
12	doubly even	13.4 µs	93 ns
30	singly even	68.0 µs	76 ns
60	doubly even	357.4 µs	99 ns
99	odd	531.3 µs	54 ns

Table 1. Measured time of the constructive algorithms (median of 5 measurements, each averaged over many repetitions).

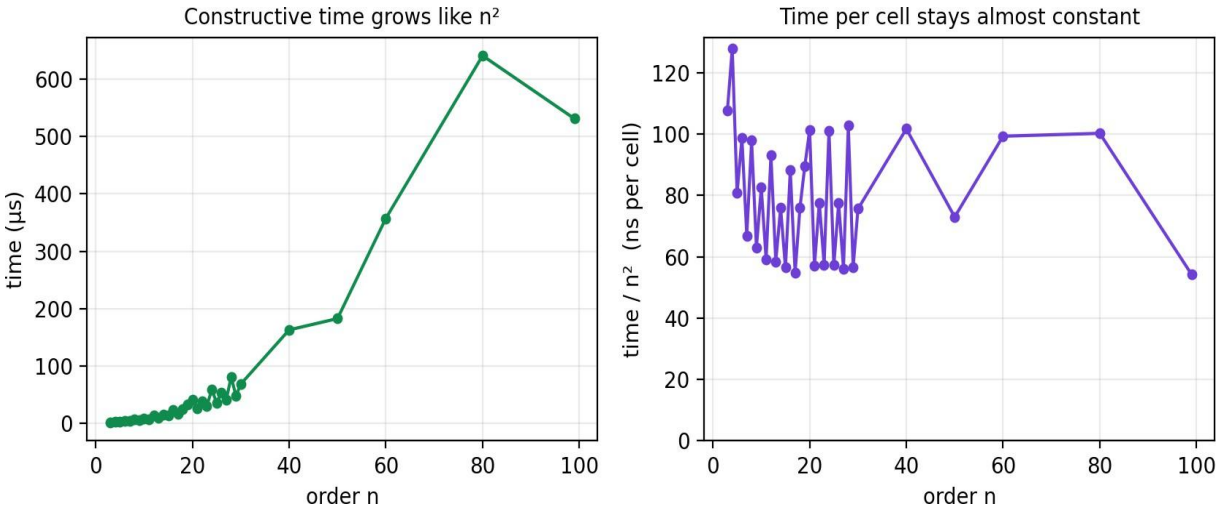


Figure . Left: the constructive time grows like n². Right: the time per cell stays almost the same, which is what Θ(n²) predicts (small changes come from memory and timer effects).

5.2. Naive brute force: O((n²)!)

There are (n²)! different arrangements of the numbers 1 … n². In the worst case the naive search must examine all of them, and testing one arrangement costs O(n) to O(n²). So the worst-case time is O((n²)! · n²), and we write the number of arrangements as O((n²)!), as in the assignment sheet. By Stirling's formula, (n²)! ≈ √(2πn²) · (n²/e)n², which grows like 2Θ(n² log n): faster than any exponential in n. The growth is shown in the table of Section 4.5.

Average case. Magic squares are rare but not absent. There are 8 magic squares of order 3 (one square and its rotations and reflections), 7,040 of order 4 (880 different squares × 8), about 2.2 × 10^9 of order 5 (275,305,224 × 8). A search that tries arrangements in random order needs about (n²)! divided by this count before it hits one.

For n = 3 that is 9!/8= 45,360 arrangements, which matches our measurement (69,075 arrangements). For larger n the average case is still enormous (Section 4.5).

5.3. Backtracking: worst case and why pruning helps:

Worst case. Backtracking explores a tree of partial grids. The number of ways to fill the first j cells is $(n^2)!/(n^2-j)!$, so if pruning never removed anything the tree would have about $e \cdot (n^2)!$ nodes. No pruning rule gives a guarantee for every input, so the worst-case time is still $O((n^2)!)$ and the search is exponential-like in n^2.

Why it is much faster in practice. Pruning cuts whole branches, not single arrangements. When a rule rejects a number in cell j, all the $(n^2-j)!$ arrangements below it are skipped without being looked at. The completion rule is especially strong: the last cell of each row is decided by the sum, so only about $(n - 1)$ free cells per row are really chosen instead of n. The overflow and reachability rules remove partial rows that cannot be finished. Our measurements show the effect: for n = 3, backtracking found a square after 632 nodes where naive search needed 69,075 arrangements, and for n = 4 it found a square after 47,564 nodes in about 22 ms, while the naive search had not finished after 35.7 million arrangements (5 s).

5.4. Measured results:

We measured the three methods with the same functions that the program's Benchmark window and Compare both button use. Each brute-force run had a time limit (5 s for backtracking and for naive n = 4, 3 s for naive n = 5). The numbers below were measured with Python 3.12 on a Linux computer; absolute times depend on the machine, but the shape of the results does not. The group can repeat the measurements on its own computer with the Benchmark window.

n	Constructive	Backtracking: result	BT nodes	Naive: result	Naive arrangements
3	1.0 µs	269.9 µs	632	13.13 ms	69,075
4	2.0 µs	22.28 ms	47,564	Limit reached (5 s)	35,656,704
5	2.0 µs	Limit reached (5 s)	9,702,400	Limit reached (3 s)	18,339,840
6	3.6 µs	Limit reached (5 s)	9,708,544	Not run (see 4.5)	—

Table . Time taken by each method. "Limit reached" means no magic square was found before the time limit.

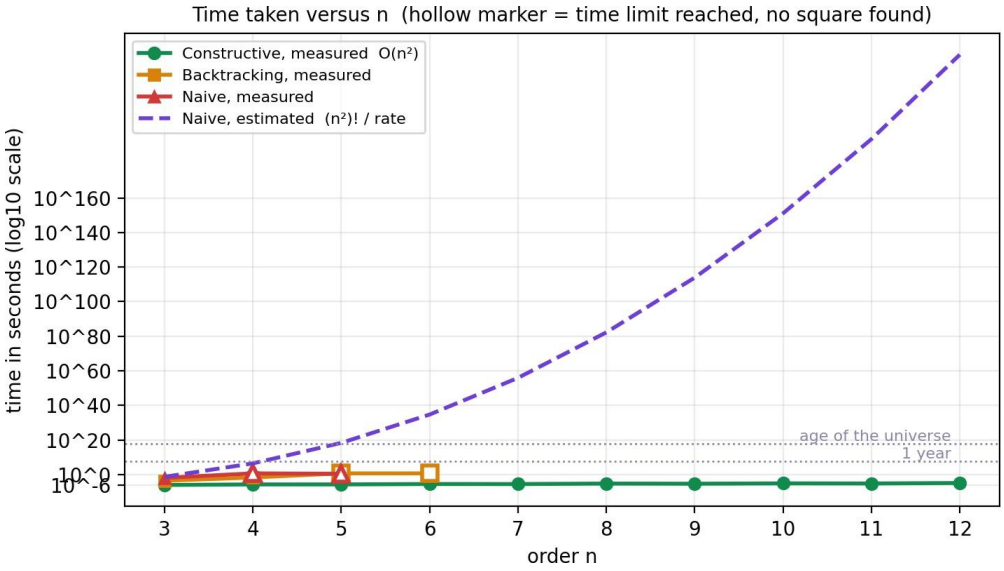


Figure . Time versus n on a logarithmic scale. The constructive time hardly rises. The brute-force lines stop where the time limit was reached. The dashed line is the estimate (n²)! ÷ measured speed. The horizontal dotted lines mark one year and the age of the universe.

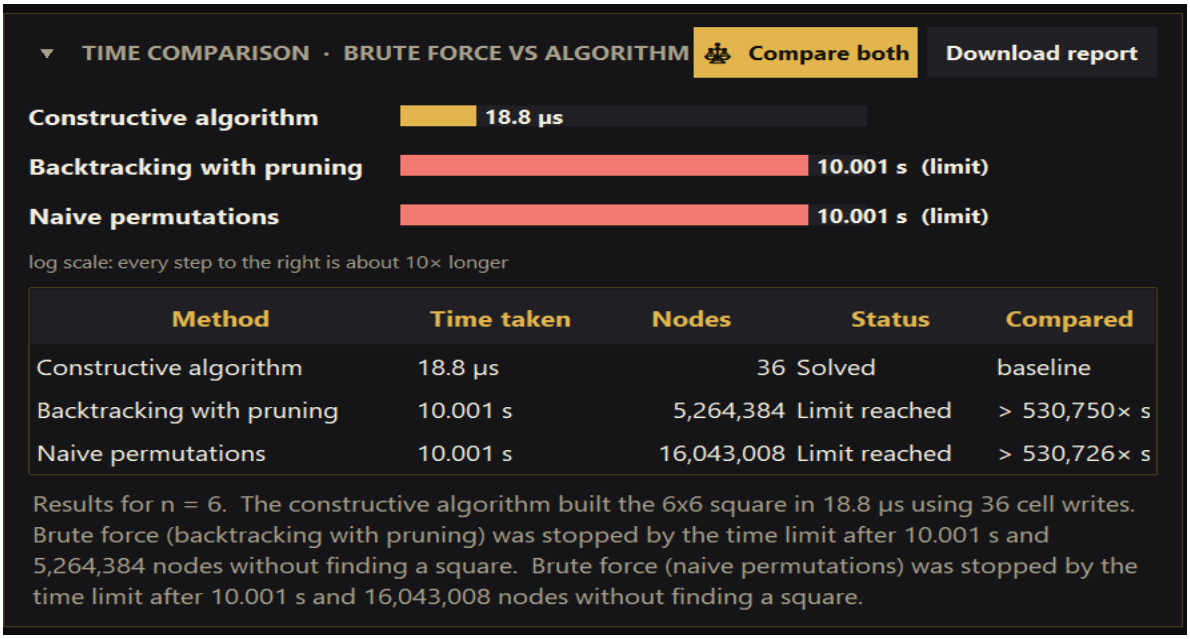


Figure 10. Time comparison in Ordematrix App.

5.5. When does brute force become impractical?

The naive search checked about 7.1 million arrangements per second in our test. We use this speed to estimate how long the worst case would take. We also estimate the average case for n = 4, 5 and 6 from the number of magic squares that exist.

n	$(n^2)!$ arrangements	Worst-case time	Magic squares	Average-case time
3	362,880	51 ms	8	6 ms
4	2.09×10^13	34 days	7,040	6.9 min
5	1.55×10^25	10^10.8 years	2,202,441,792	31 years
6	3.72×10^41	10^27.2 years	≈ 1.4e+20	10^7.1 years
7	6.08×10^62	10^48.4 years	unknown	huge
8	1.27×10^89	10^74.8 years	unknown	huge

Table . Estimated time of the naive search at the measured speed.

- **n = 3** is easy for every method.
- **n = 4:** naive search is already slow. The worst case is about a month. Backtracking solves it in a fraction of a second, so it is the largest order where brute force is practical.
- **n = 5** and above: naive search is hopeless (the worst case is more than five times the age of the universe), and even the average case takes decades. In our test backtracking.

examined about 9.7 million nodes in 5 s for n = 5 and n = 6 without finding a square, so it also hits the limit. The constructive algorithm builds a 99 × 99 square (9,801 numbers) in about a millisecond or less than a few milliseconds in our test.

5.6. Summary of the complexities:

Method	Worst-case time	Cost per unit of work	Space	Why	Practical limit
Siamese (odd)	$\Theta(n^2)$	O(1) per cell	$O(n^2)$	one constant-time step per cell	any n (tested to 99)
Doubly even	$\Theta(n^2)$	O(1) per cell, two passes	$O(n^2)$	two passes over the cells	any n (tested to 99)
Strachey (singly even)	$\Theta(n^2)$	O(1) per cell	$O(n^2)$	odd square + copy + swaps	any n (tested to 98)
Naive brute force	$O((n^2)! \cdot n^2)$, usually written $O((n^2)!)$	O(n) per arrangement with the first-row test, up to $O(n^2)$ for the full check	$O(n^2)$	tries every permutation	n = 3
Backtracking with pruning	$O((n^2)!)$, much less in practice	O(n) to $O(n^2)$ per node	$O(n^2)$	prunes whole branches early	n = 3, 4

6. Challenges:

- ***Even orders, especially the singly even case:***

  The odd method is simple, but even orders are harder. The singly even case has a special rule for the middle row (the shifted swap), and one wrong column gives a square that looks right but fails on one diagonal. We solved this in three ways: we followed the exact rules from the sources [3][4]; we wrote an independent checker that does not share code with the program's verifier; and we tested every order from 3 to 60 plus 98 and 99. We also compared our 6 × 6 result with the known Strachey example, and they are identical.

- ***Two conventions for the doubly even case:***

  The assignment sheet complements the diagonal cells, but our notes complement the other cells. At first the 4 × 4 results looked different and we thought one was wrong. We then showed that one square is the complement of the other, so both are correct. We follow the assignment sheet and explain the difference in Section 2.2.

- ***Keeping the window responsive:***

  Our first idea of running the search inside a button handler would freeze the window. We moved the search to a worker thread. Because Tkinter widgets may only be changed by the main thread, the worker never touches them; it only updates a counter and a queue, and the GUI reads them with timers. To stop a deep recursive search we check a cancel flag and the clock every 1,024 nodes.

- ***Brute force that never ends:***

  For $n \geq 4$ the naive search cannot finish, so we added a user-set time limit, a Stop button, and the clear status “Limit reached”. The Compare both button uses the same limit, so the comparison always ends.

- ***Measuring very small times:***

  The constructive algorithms take only microseconds, so one measurement is mostly noise. We repeat each construction many times and use the average, and for the report we took the median of five such averages.

- ***GUI Quality:***

  The first version of the screen looked blurry on Windows because the program was not “DPI aware”. We fixed this by enabling DPI awareness before the window opens. Later we found, by running the program, that some panels were cut off, the strategy drop-down was too small, the grid cells were too large and the colour-theme button was hidden under another button. We fixed each problem step by step: a scrollable page, compact spacing, smaller cells and a better button layout. We also made all colours “tokens” so the whole interface can change between the Black & Gold and Light Violet themes.

- ***Exporting without extra libraries:***

  We wanted PDF export without asking the user to install anything. We wrote a small PDF writer using the standard fonts. A few characters, such as ² and µ, are not available there, so the PDF shows ^2 and u instead.

- ***AI-generated code needs testing:***

The assignment warns that AI output often contains bugs. The AI could not run or see our window, so layout problems appeared only when we ran the program. For the algorithms, our test file and the built-in verification gave us proof that every construction is correct.

7. AI usage reflection:

This was an AI-assisted assignment. We used Claude (Anthropic) [11] mainly to help with the program code. The full details, the prompts, and what we accepted, changed or rejected are in the separate file AILOG.md that is part of our submission.

How AI helped:

- It wrote the GUI code from our layout sketch and our feedback: themes, scrollable page, spin boxes, tooltips, the panel for comparing times, and the CSV / PDF / TXT export.
- It fixed the blurry display on Windows (DPI awareness).
- It helped in designing the whole overall layout of the simulator. We fixed the simulator logic and layout exactly 10 times, this involve checking of logic, smooth execution , workable buttons and completely and easily visible magic square.
- It helped the redesigning of every component according to need. Like when the program is runned some function to parts i.e. next step, algorithm, verification, time comparison needed to be scroll to see details. But after hours of thinking a new layout I come up with a plan to make every section Auto-Scrollable mean when the magic square is getting build it will automatically move to that section which step is getting executed.
- It was fast: a complete new screen in one step, and then small changes within minutes.

Where it was wrong or not good enough:

- It cannot see the screen. It produced panels that were too small, a drop-down that was squeezed, cells that were too big and a button hidden under another one. We found each problem by running the program and fixed it by describing it again, with screenshots.
- Its first design opened the time comparison in a separate window only after pressing a button. We asked for a panel that is always visible and fills in after a normal run.
- In the PDF export it first used characters (² and μ) that the built-in font cannot print.
- Our first screens were too crowded or too spread out. Good spacing needed several rounds of feedback.
- It was not very understanding to incorporate everything asked to do in a prompt at the same time.

What we learned:

- Why the three kinds of order need three different algorithms, and how to prove that the constructive methods are $\Theta(n^2)$.
- How big $(n^2)!$ really is, and why pruning helps a lot in practice but cannot change the worst case.

- How to keep a GUI responsive with threads and queues.
- That AI is a fast helper, but testing is what shows the real problems. We had to check every result ourselves, and we are able to explain every part of the submitted code.

Declaration. *As the assignment requires, we declare all AI use. Claude was the AI tool used for the code. In addition, an AI assistant(CHAT GPT) helped us to organize and execute the code and fix logical and syntax error.*

8. Conclusion:

We built Ordematrix, a Python application that constructs and simulates magic squares of every order from 3 upwards with the Siamese method (odd), the diagonal / complement method (doubly even) and Strachey's method (singly even). All three methods use $\Theta(n^2)$ time, which is the best possible because the square has n^2 cells. We tested all orders from 3 to 60 and also 98 and 99, and every square passed the independent check.

We also implemented two brute-force searches. Naive search is only practical for n = 3: it checked 69,075 arrangements in about 13 ms, but for n = 4 the worst case is about a month, and for n = 5 it is far longer than the age of the universe. Backtracking with pruning is much better, solving n = 3 in 632 nodes and n = 4 in about 22 ms, but it is still exponential in the worst case and did not finish n = 5 in 5 seconds. The constructive algorithms built a 99 × 99 square in a few milliseconds. The results show clearly why “knowing the method” beats “searching”.

A good next step would be to add Conway's LUX method as a second option for singly even orders [6], or to make the backtracking faster with smarter ordering of the cells.

Contribution Statement:

This project was completed by a single member, so the group size is one.

Member	Registration no.	Contribution
Raja Aashir Mehmood	CIIT/SP25-BED-079/ISB	All work: choosing the requirements and the design of the application, building the program with AI assistance (declared in Section 8), reviewing, running and testing it, taking the measurements with the Benchmark and Compare both, taking the screenshots, and writing the report (Assignment 2), the README and the AI log.

References

1. E. W. Weisstein, "Magic Square," *MathWorld – A Wolfram Web Resource*. [Online]. Available: https://mathworld.wolfram.com/MagicSquare.html (accessed Oct. 2026).
2. D. Hawley, "Magic squares," *NRICH*, University of Cambridge. [Online]. Available: https://nrich.maths.org/1337 (accessed Oct. 2026).
3. L. M. Leite, V. Jacquemin and N. Boillot, "Magic Squares," Experimental Mathematics report, Univ. of Luxembourg, 2015/2016, Secs. 5.2.1–5.2.2. [Online]. Available: https://math.uni.lu/eml/assets/reports/2016/Magic_Squares.pdf (accessed Oct. 2026).
4. R. K. S. Hankin, "strachey: Strachey's algorithm for magic squares," R package *magic*, R Documentation. [Online]. Available: https://search.r-project.org/CRAN/refmans/magic/html/strachey.html (accessed Oct. 2026).
5. "Strachey method for magic squares," *Wikipedia*. [Online]. Available: https://en.wikipedia.org/wiki/Strachey_method_for_magic_squares (accessed Oct. 2026).
6. E. W. Weisstein, "LUX Method," *MathWorld*. [Online]. Available: https://mathworld.wolfram.com/LUXMethod.html (accessed Oct. 2026).
7. A. Umar, "On the construction of even order magic squares," arXiv:1202.0948. [Online]. Available: https://arxiv.org/abs/1202.0948
8. A. M. Ibrahim, H. M. Jibril and A. Umar, "Constructing even order magic squares by consecutive numbering," arXiv:1303.4536. [Online]. Available: https://arxiv.org/abs/1303.4536
9. Python Software Foundation, "The Python 3 documentation: tkinter, itertools, threading, queue, time." [Online]. Available: https://docs.python.org/3/ (accessed Oct. 2026).
10. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
11. Anthropic, "Claude" (large language model). [Online]. Available: https://claude.ai (used Oct. 2026 for program code, tests and report review; see AI_LOG.md).
12. OpenAI, "ChatGPT" (large language model). [Online]. Available: https://chat.openai.com (used [month] 2026 for [task]; see AI_LOG.md). Keep this only if you really used it.
13. Levitin, *Introduction to the Design and Analysis of Algorithms*, 3rd ed., Pearson, 2012
14. K. Pinn and C. Wieczerkowski, 1998, *Number of magic squares from parallel tempering Monte Carlo*